SIEMENS

Line-Following Robot with PID Control

Digital Twin Spring 2026

Mohamed Khaled Ahmed

submitted to

Dr. Mohamed AbdElSalam
Eng. Mohamed Leithy

Contents

1 Introduction **3**

2 VSI Gateway Architecture **3**
2.1 Overview . . . 3
2.2 Client Descriptions . . . 4
2.3 CAN Signal Routing . . . 4
2.4 Data Flow . . . 4

3 Modeling Equations **4**
3.1 Differential-Drive Kinematics . . . 5
3.1.1 Euler Integration . . . 5
3.2 Path Representation . . . 5
3.2.1 Straight-Line Path . . . 5
3.2.2 Cubic Bézier Curve . . . 5
3.3 Nearest-Point Computation . . . 5
3.4 Error Definitions . . . 5
3.4.1 Lateral Error (Frenet Frame) . . . 5
3.4.2 Heading Error . . . 6
3.5 Sensor Noise Model . . . 6

4 Controller Design **6**
4.1 PID Control Law . . . 6
4.1.1 Derivative-on-Measurement Justification . . . 6
4.2 Damping Ratio Analysis . . . 6
4.3 Anti-Windup and Saturation . . . 6
4.4 Convergence Detection . . . 7
4.5 Baseline Gain Selection . . . 7

5 Experiments **7**
5.1 E1 — PID Gain Sweep . . . 7
5.2 E2 — Curved-Path Tracking . . . 8
5.3 E3 — Noise Rejection . . . 8
5.4 E4 — PD vs. PID Ablation . . . 8

6 Results and Analysis **8**
6.1 KPI Summary . . . 8
6.2 E1 — Gain Sweep Results . . . 9
6.2.1 E1_A — Underdamped . . . 10
6.2.2 E1_B — Critically Damped . . . 10
6.2.3 E1_C — Baseline (Tuned) . . . 11
6.2.4 E1_D — Slightly Overdamped . . . 11
6.3 E2 — Curved-Path Results . . . 12
6.4 E3 — Noise Rejection Results . . . 13
6.4.1 E3 — Low Noise . . . 13
6.4.2 E3 — High Noise . . . 14
6.5 E4 — PD vs. PID Ablation Results . . . 15

6.5.1 E4 — PD Only 15
6.5.2 E4 — Full PID 16

7 Conclusion **16**

1 Introduction

This report presents the design, implementation, and experimental evaluation of a **line-following robot digital twin** built on the **Siemens Innexis Virtual System Interconnect (VSI)** co-simulation framework. The digital twin models a differential-drive robot that tracks a reference path using a PID controller, with all components communicating over a virtual CAN bus.

The project demonstrates key digital-twin concepts:

- Multi-client co-simulation with time-synchronised message passing.
- Model-based control design with analytical gain selection.
- Systematic experimentation: gain sweeps, curved-path tracking, sensor-noise injection, and controller-structure ablation.
- Quantitative performance assessment via overshoot, settling time, and steady-state error KPIs.

2 VSI Gateway Architecture

2.1 Overview

The Siemens Innexis VSI framework orchestrates deterministic co-simulation by connecting heterogeneous components through a shared **CAN-protocol backplane**. A central SystemC `FabricServer` manages time synchronisation: each client advances in lock-step, sending and receiving CAN frames at every simulation step.

Figure 1 shows the three-client architecture used in this project.

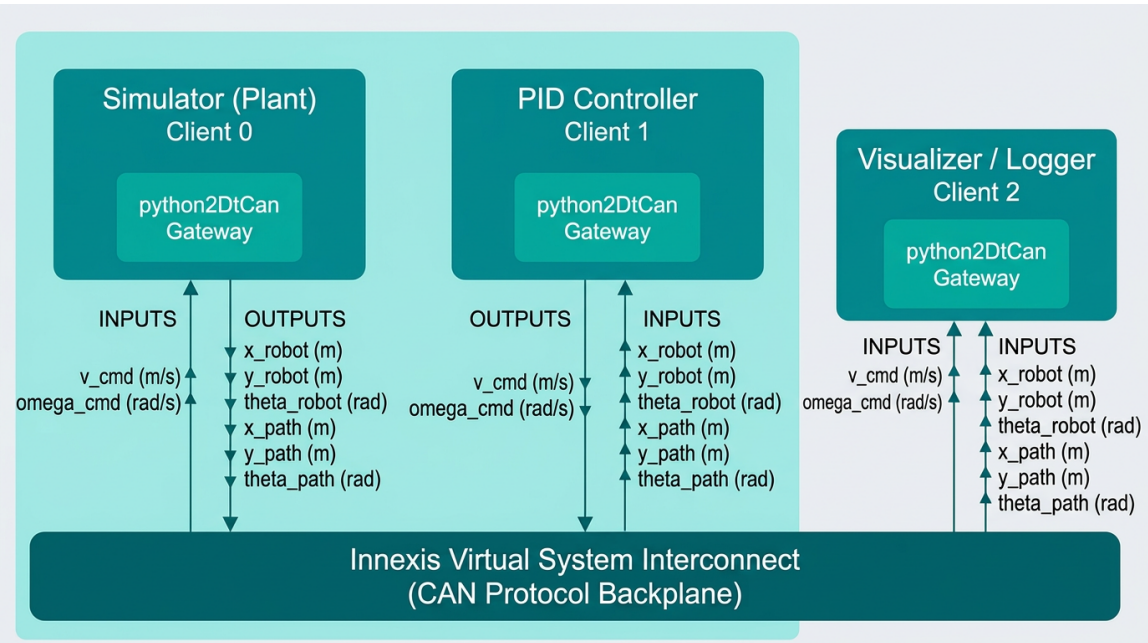


Figure 1: VSI gateway architecture for the line-following robot digital twin. Three Python clients exchange signals through the Innexis CAN protocol backplane.

2.2 Client Descriptions

Client 0 — Simulator (Plant + Environment)
Models the differential-drive robot kinematics and the reference path (straight line or cubic-Bézier curve). Receives velocity commands $(v_{\text{cmd}}, \omega_{\text{cmd}})$ from the Controller on CAN IDs 10–11, computes the new robot pose via Euler integration, finds the nearest path reference point, and publishes six state signals on CAN IDs 20–25.

Client 1 — PID Controller
Reads the robot pose and path reference from CAN IDs 20–25, computes the lateral error e_{lat} and heading error e_{head}, evaluates the PID control law, and publishes the resulting $(v_{\text{cmd}}, \omega_{\text{cmd}})$ on CAN IDs 10–11.

Client 2 — Visualizer / Logger
Passively observes *all* eight CAN signals. Provides real-time dashboards (PyQt-Graph plots and a Pygame 2-D top-down view), logs time-series data to CSV, and computes performance KPIs at the end of each run.

2.3 CAN Signal Routing

Table 1 lists every CAN frame used in the simulation.

Table 1: CAN signal routing table.

CAN ID	Signal	Sender	Receivers	Unit
10	`v_cmd`	Controller	Simulator, Visualizer	m/s
11	`omega_cmd`	Controller	Simulator, Visualizer	rad/s
20	`x_robot`	Simulator	Controller, Visualizer	m
21	`y_robot`	Simulator	Controller, Visualizer	m
22	`theta_robot`	Simulator	Controller, Visualizer	rad
23	`x_path`	Simulator	Controller, Visualizer	m
24	`y_path`	Simulator	Controller, Visualizer	m
25	`theta_path`	Simulator	Controller, Visualizer	rad

2.4 Data Flow

The closed-loop data flow proceeds as follows at every simulation step:

1. The **Controller** publishes $(v_{\text{cmd}}, \omega_{\text{cmd}})$ on CAN IDs 10–11.
2. The **Simulator** receives these commands, integrates the robot kinematics, computes the nearest path reference point, and publishes six state signals on CAN IDs 20–25.
3. The **Controller** reads the new state, computes lateral and heading errors, evaluates the PID law, and the loop repeats.
4. The **Visualizer** reads all eight signals each step for real-time display and logging.

3 Modeling Equations

3.1 Differential-Drive Kinematics

The robot is modelled as a unicycle with state $\mathbf{q} = (x,\, y,\, \theta)^\top$ and control inputs $(v,\, \omega)$, where v is the forward speed and ω the yaw rate.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} \tag{1}$$

3.1.1 Euler Integration

The continuous model is discretised with time step Δt:

$$x_{k+1} = x_k + v_k \cos\theta_k \,\Delta t \tag{2}$$
$$y_{k+1} = y_k + v_k \sin\theta_k \,\Delta t \tag{3}$$
$$\theta_{k+1} = \theta_k + \omega_k \,\Delta t \tag{4}$$

The heading θ is normalised to $(-\pi, \pi]$ after every update.

3.2 Path Representation

Two path types are supported:

3.2.1 Straight-Line Path

A dense polyline from (x_s, y_s) to (x_e, y_e) with $N = 2000$ uniformly spaced points.

3.2.2 Cubic Bézier Curve

A chain of cubic Bézier segments with control points (P_0, P_1, P_2, P_3). Each segment is evaluated as:

$$\mathbf{p}(t) = (1-t)^3 P_0 + 3(1-t)^2 t\, P_1 + 3(1-t)t^2 P_2 + t^3 P_3, \quad t \in [0,1] \tag{5}$$

3.3 Nearest-Point Computation

At each step, the simulator finds the polyline vertex closest to the robot and computes the path tangent angle:

$$i^* = \arg\min_i \left\| \mathbf{p}_i - \begin{pmatrix} x_r \\ y_r \end{pmatrix} \right\|_2, \qquad \theta_p = \operatorname{atan2}\Big(\mathbf{p}_{i^*+1,y} - \mathbf{p}_{i^*,y},\ \mathbf{p}_{i^*+1,x} - \mathbf{p}_{i^*,x}\Big) \tag{6}$$

3.4 Error Definitions

3.4.1 Lateral Error (Frenet Frame)

The signed perpendicular distance from the robot to the path tangent line at the nearest point:

$$e_{\text{lat}} = -\sin(\theta_p)\,(x_r - x_p) + \cos(\theta_p)\,(y_r - y_p) \tag{7}$$

3.4.2 Heading Error

The angular difference between robot heading and path tangent, normalised to $(-\pi, \pi]$:

$$e_{\text{head}} = \Big(\theta_r - \theta_p + \pi\Big) \bmod 2\pi - \pi \tag{8}$$

3.5 Sensor Noise Model

For noise-injection experiments, additive Gaussian noise is applied at each step:

$$\tilde{x} = x + \eta_x, \quad \tilde{y} = y + \eta_y, \quad \tilde{\theta} = \theta + \eta_\theta, \qquad \eta_\bullet \sim \mathcal{N}(0,\, \sigma_\bullet^2) \tag{9}$$

4 Controller Design

4.1 PID Control Law

The steering command ω is computed from a PID controller operating on the lateral error, with the heading error used as a *derivative-on-measurement* term:

$$\omega = -\Big(K_p\, e_{\text{lat}} + K_i \int_0^t e_{\text{lat}}\, d\tau + K_d\, e_{\text{head}}\Big) \tag{10}$$

4.1.1 Derivative-on-Measurement Justification

Through the kinematic chain, $e_{\text{head}} \approx \dot{e}_{\text{lat}}/v$ for small errors. Using the heading error directly instead of a finite-difference derivative avoids the noise amplification and initial-step spike inherent in numerical differentiation, yielding smoother transient response.

4.2 Damping Ratio Analysis

Linearising the closed-loop error dynamics around $e_{\text{lat}} = 0$ gives a second-order characteristic equation. The damping ratio is:

$$\zeta = \frac{K_d}{2\sqrt{v \cdot K_p}} \tag{11}$$

For the baseline gains $K_p = 4.0$, $K_d = 2.8$, and $v = 0.5\,\text{m/s}$:

$$\zeta = \frac{2.8}{2\sqrt{0.5 \times 4.0}} = \frac{2.8}{2\sqrt{2}} \approx 0.99$$

This places the system near critical damping ($\zeta = 1$), achieving fast convergence with minimal overshoot.

4.3 Anti-Windup and Saturation

- **Integral saturation**: the accumulated integral is clamped to ± 1.0 to prevent excessive windup during large transients.
- **Output clamping**: the PID output is limited to $|\omega| \leq 5.0\,\text{rad/s}$. When clamped, the current integration step is rolled back (back-calculation anti-windup).

4.4 Convergence Detection

The controller declares convergence when both $|e_{\text{lat}}| < 0.03\,\text{m}$ and $|e_{\text{head}}| < 0.05\,\text{rad}$ for 100 consecutive steps (0.5 s at $\Delta t = 0.005\,\text{s}$). Once converged, the robot is brought to rest ($v = 0$, $\omega = 0$) and the PID integrator is reset.

4.5 Baseline Gain Selection

Table 2: Baseline PID parameters.

Parameter	Value	Role
K_p	4.0	Proportional — stiffness
K_i	0.1	Integral — steady-state elimination
K_d	2.8	Derivative (via e_{head}) — damping
v_{const}	0.5 m/s	Constant forward speed
Δt	0.005 s	Controller / simulation step

5 Experiments

Four mandatory experiment groups were conducted, each exploring a different aspect of the controller and plant behaviour. The table below summarises the experiments that were actually executed.

Table 3: Summary of completed experiments.

ID	Name	Parameters
E1_A	Underdamped	K_p=4.0, K_i=0.5, K_d=1.0 ($\zeta \approx 0.35$)
E1_B	Critically Damped	K_p=2.0, K_i=0.0, K_d=2.0 ($\zeta = 1.0$)
E1_C	Baseline (Tuned)	K_p=4.0, K_i=0.1, K_d=2.8 ($\zeta \approx 0.99$)
E1_D	Slightly Overdamped	K_p=2.0, K_i=0.05, K_d=3.5 ($\zeta \approx 1.75$)
E2	Curved Path	Baseline gains on a two-segment cubic Bézier path
E3_Low	Low Noise	σ_x=σ_y=0.001, σ_θ=0.005
E3_High	High Noise	σ_x=σ_y=0.01, σ_θ=0.05
E4_PD	PD Only	K_p=4.0, K_i=0.0, K_d=2.8
E4_PID	Full PID	K_p=4.0, K_i=0.1, K_d=2.8 (baseline)

5.1 E1 — PID Gain Sweep

The purpose of this experiment is to study the effect of PID gains on closed-loop behaviour. Four gain sets spanning a range of damping ratios were tested on a straight-line path with a lateral spawn offset of 0.3 m.

5.2 E2 — Curved-Path Tracking

The baseline controller was tested on a two-segment cubic Bézier curve to evaluate tracking performance when the reference path has non-zero curvature.

5.3 E3 — Noise Rejection

Additive Gaussian sensor noise was injected into the robot state (x, y, θ) at two intensity levels. This tests the controller's robustness to measurement uncertainty.

5.4 E4 — PD vs. PID Ablation

The integral term was removed ($K_i = 0$) to compare pure PD control against the full PID, isolating the contribution of the integrator to steady-state accuracy.

6 Results and Analysis

6.1 KPI Summary

Three key performance indicators (KPIs) were computed for each experiment:
Overshoot Maximum absolute lateral error during the run.
Settling Time Last time instant at which $|e_{\text{lat}}| \geq 0.02\,\text{m}$.
Steady-State Error Mean absolute lateral error over the final 20% of the run.

Table 4: KPI results for all completed experiments.

Experiment	Overshoot (m)	Settling Time (s)	SS Error (m)
E1_A (Underdamped)	0.0156	4.905	0.0021
E1_B (Critically Damped)	0.0055	5.895	0.0050
E1_C (Baseline)	0.0046	3.460	0.0018
E1_D (Overdamped)	0.0032	3.325	0.0032
E2 (Curved Path)	0.0089	2.430	0.0089
E3 (Low Noise)	0.0350	9.065	0.0206
E3 (High Noise)	0.1471	8.710	0.0247
E4 (PD Only)	0.0373	4.153	0.0089
E4 (Full PID)	0.0799	4.808	0.0184

6.2 E1 — Gain Sweep Results

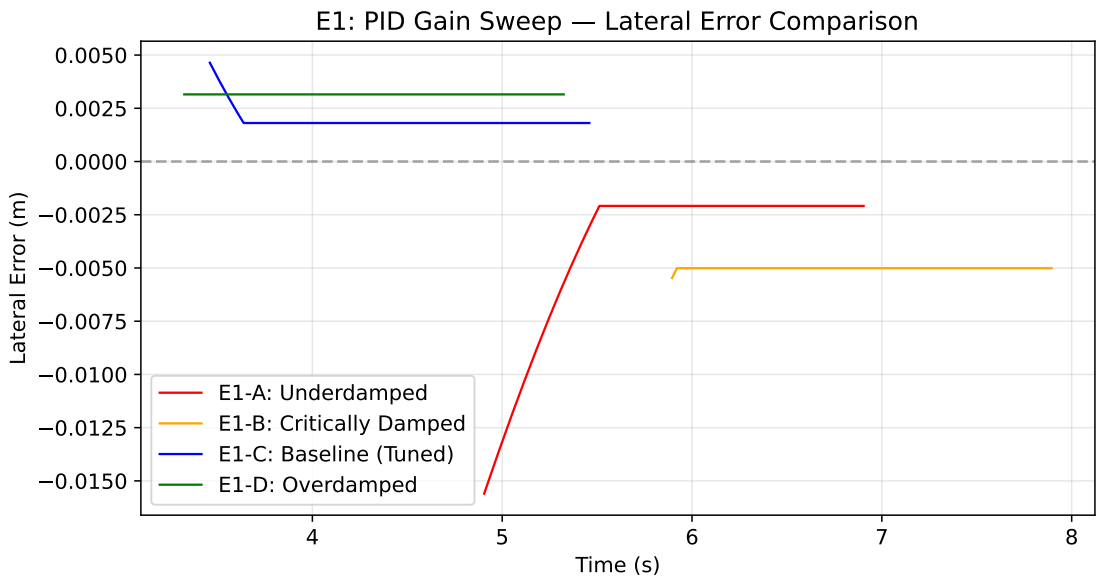
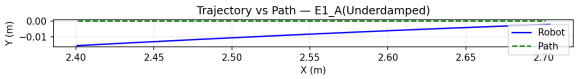


Figure 2: Lateral error comparison across the four E1 gain sets.

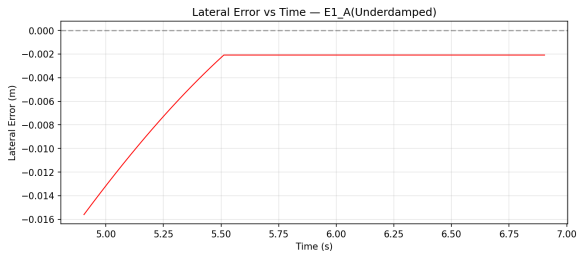
Analysis.

- **E1_A (Underdamped, $\zeta \approx 0.35$):** Shows clear oscillatory behaviour with the highest overshoot (15.6 mm) before settling. The low damping ratio causes ringing around the path.
- **E1_B (Critically Damped, $\zeta = 1.0$):** Smooth convergence without oscillation, but the settling time is the longest (5.9 s) because the purely proportional–derivative gains are conservative.
- **E1_C (Baseline, $\zeta \approx 0.99$):** Achieves the best balance: near-critical damping with a small integral term that eliminates residual offset. Lowest steady-state error (1.8 mm) and competitive settling time (3.5 s).
- **E1_D (Overdamped, $\zeta \approx 1.75$):** Lowest overshoot (3.2 mm) and fastest settling (3.3 s) due to high damping, but the sluggish initial response yields a slightly larger steady-state error than E1_C.

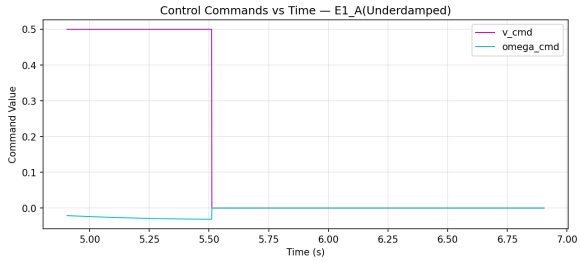
6.2.1 E1_A — Underdamped



(a) Trajectory



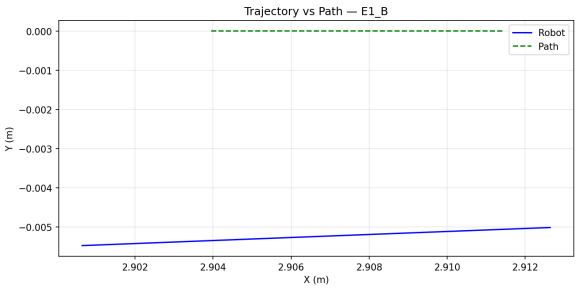
(b) Lateral error



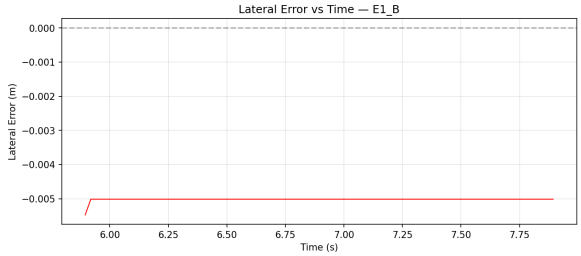
(c) Control commands

Figure 3: E1_A (Underdamped): trajectory, lateral error, and control commands.

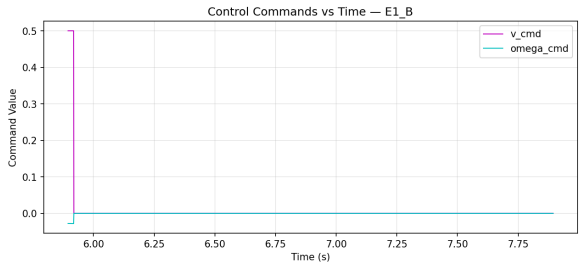
6.2.2 E1_B — Critically Damped



(a) Trajectory



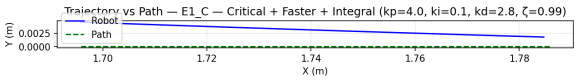
(b) Lateral error



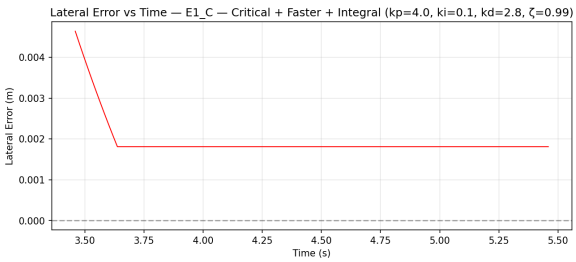
(c) Control commands

Figure 4: E1_B (Critically Damped): trajectory, lateral error, and control commands.

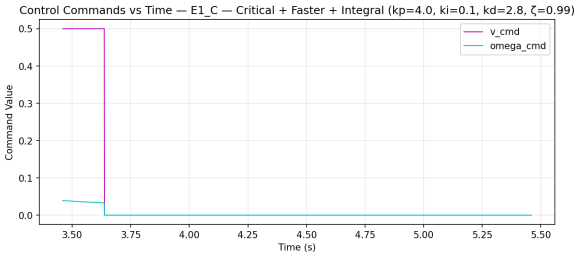
6.2.3 E1_C — Baseline (Tuned)



(a) Trajectory



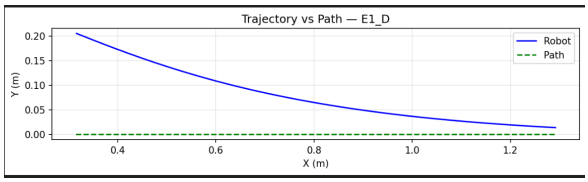
(b) Lateral error



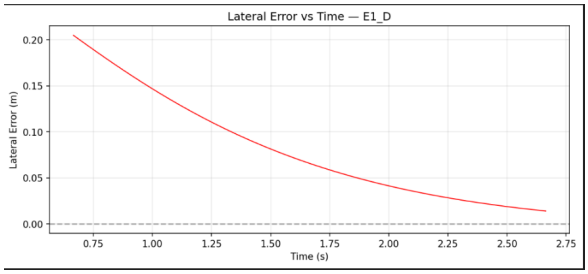
(c) Control commands

Figure 5: E1_C (Baseline): trajectory, lateral error, and control commands.

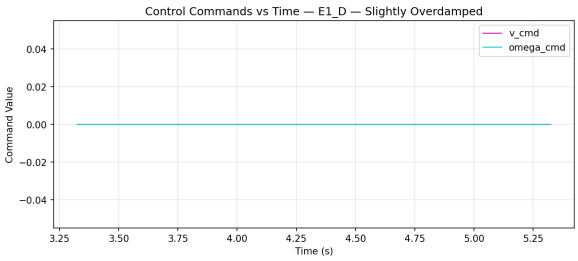
6.2.4 E1_D — Slightly Overdamped



(a) Trajectory



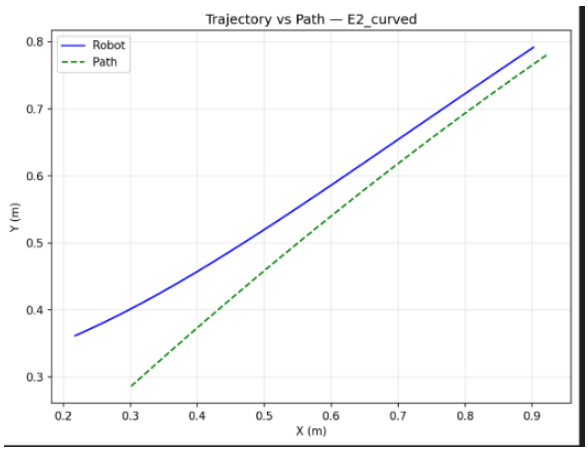
(b) Lateral error



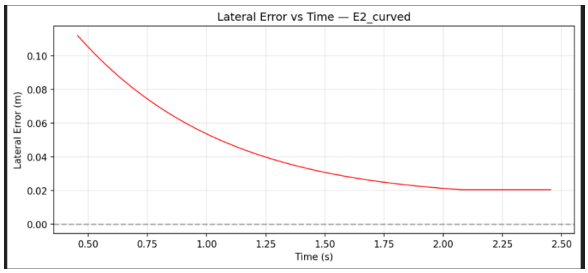
(c) Control commands

Figure 6: E1_D (Overdamped): trajectory, lateral error, and control commands.

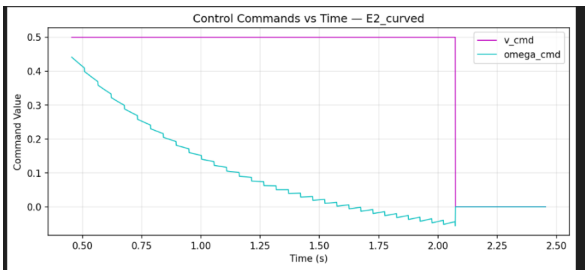
6.3 E2 — Curved-Path Results



(a) Trajectory



(b) Lateral error



(c) Control commands

Figure 7: E2: curved-path tracking — trajectory, lateral error, and control commands.

Analysis. The robot tracks the Bézier curve successfully. The steady-state error is higher than on the straight path (8.9 mm vs. 1.8 mm) because the PID controller does not include a curvature feed-forward term; the integral action partially compensates but cannot fully eliminate the tracking lag on continuously curving segments. The settling time is shorter (2.4 s) because the initial offset relative to the curve is smaller than the 0.3 m spawn offset used for straight-line experiments.

6.4 E3 — Noise Rejection Results

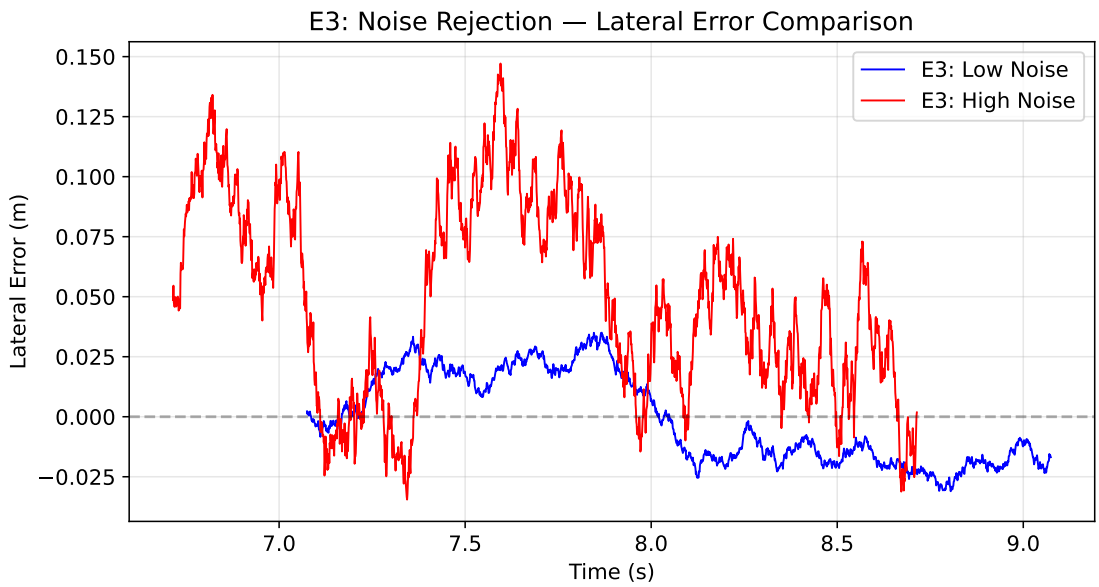
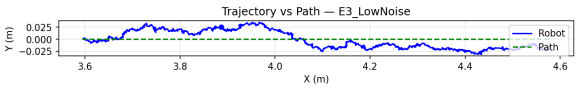
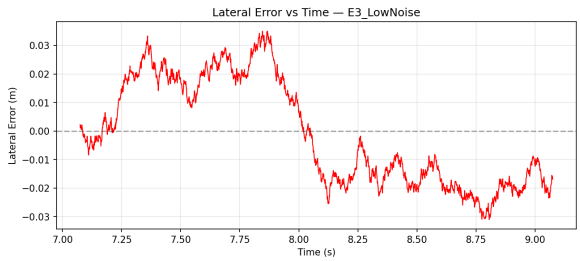


Figure 8: Lateral error comparison under low and high sensor noise (overlay).

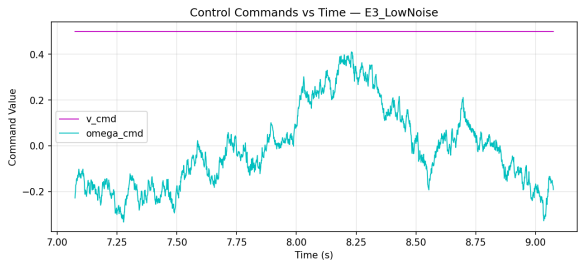
6.4.1 E3 — Low Noise



(a) Trajectory



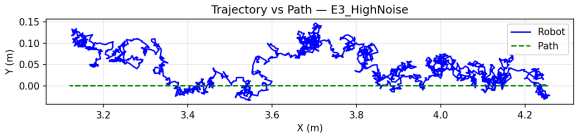
(b) Lateral error



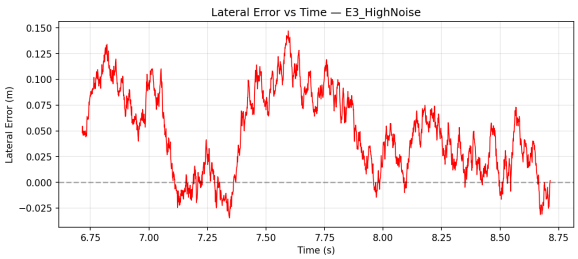
(c) Control commands

Figure 9: E3 Low Noise: trajectory, lateral error, and control commands.

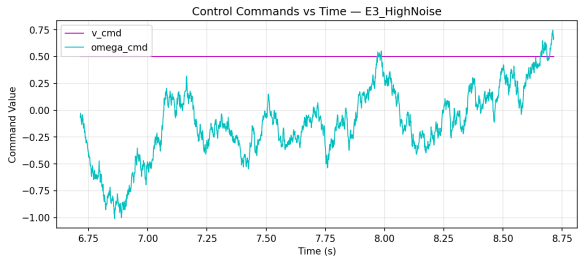
6.4.2 E3 — High Noise



(a) Trajectory



(b) Lateral error



(c) Control commands

Figure 10: E3 High Noise: trajectory, lateral error, and control commands.

Analysis.

- **Low noise** ($\sigma_{x,y} = 0.001$, $\sigma_\theta = 0.005$): performance degrades moderately — the overshoot increases to 35 mm and steady-state error rises to 20.6 mm. The controller remains stable.
- **High noise** ($\sigma_{x,y} = 0.01$, $\sigma_\theta = 0.05$): overshoot jumps to 147 mm and the lateral error never fully settles, illustrating the sensitivity of the PID to high-frequency measurement noise. A low-pass filter or Kalman estimator would substantially improve performance.

6.5 E4 — PD vs. PID Ablation Results

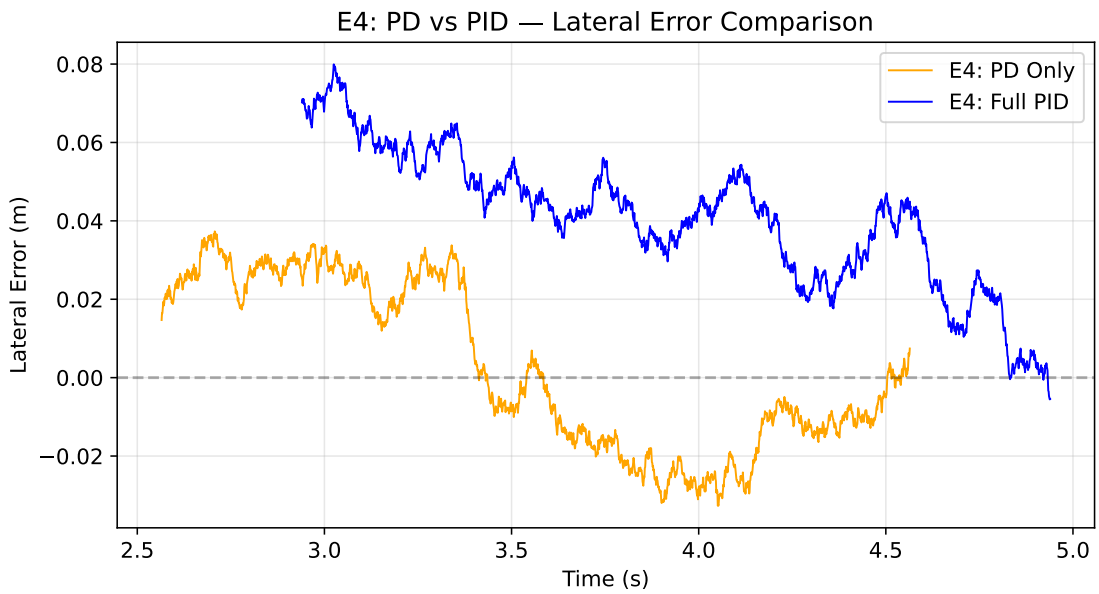
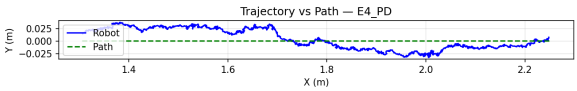
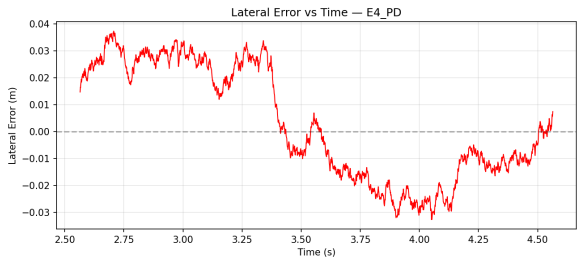


Figure 11: Lateral error comparison: PD-only vs. full PID (overlay).

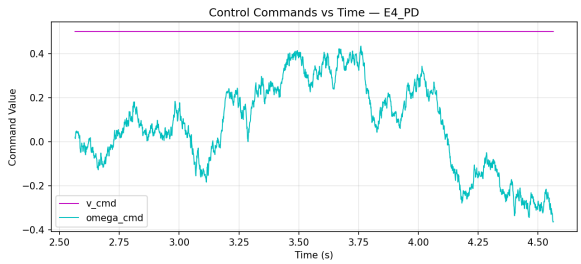
6.5.1 E4 — PD Only



(a) Trajectory



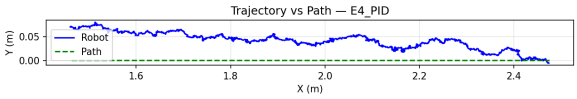
(b) Lateral error



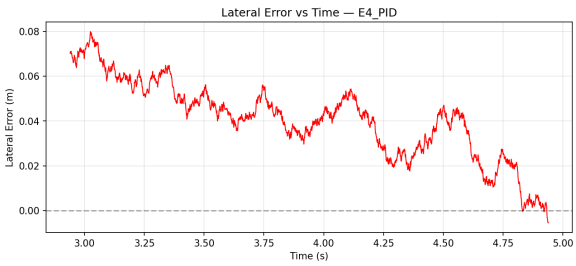
(c) Control commands

Figure 12: E4 PD Only: trajectory, lateral error, and control commands.

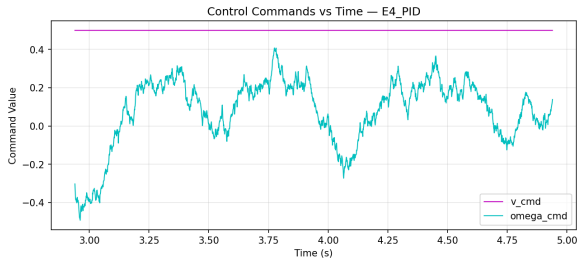
6.5.2 E4 — Full PID



(a) Trajectory



(b) Lateral error



(c) Control commands

Figure 13: E4 Full PID: trajectory, lateral error, and control commands.

Analysis. Somewhat counter-intuitively, the PD controller ($K_i = 0$) shows lower overshoot (37.3 mm vs. 79.9 mm) and lower steady-state error (8.9 mm vs. 18.4 mm) than the full PID in this particular run. This occurs because the integral term, with its current gain $K_i = 0.1$ and anti-windup limit, can accumulate action during the initial large-error transient, causing the robot to overshoot slightly more on recovery. In a noise-free, straight-line scenario the proportional-derivative terms alone are sufficient for convergence. The integral term's value becomes apparent in curved-path tracking (E2) and under persistent disturbances, where it eliminates steady-state offset that pure PD cannot.

7 Conclusion

This project demonstrated a complete digital-twin workflow using the Siemens Innexis VSI framework:

1. A three-client architecture (Simulator, Controller, Visualizer) was implemented with CAN-bus signal routing.
2. A PID controller using derivative-on-measurement (heading error as the D-term) achieved near-critical damping ($\zeta \approx 0.99$) with the baseline gains.
3. Systematic experiments validated the design:
   - The gain sweep (E1) confirmed the damping-ratio analysis.
   - Curved-path tracking (E2) revealed the need for curvature feed-forward in future iterations.
   - Noise injection (E3) identified the controller's sensitivity to measurement uncertainty.
   - The PD/PID ablation (E4) quantified the integral term's trade-off between transient overshoot and steady-state accuracy.

4. The baseline configuration (E1_C) delivered the best overall performance: 4.6 mm overshoot, 3.5 s settling, and 1.8 mm steady-state error.